Exercício Final: Análise de Dados com OpenMP

Coeficiente de variação para dados agrupados em classes · Programação Paralela (CCO085) · IESB

Alunos: João Otavio Troiano Segoria e Bruno Nóbrega Souza

1. O que o programa faz

O programa `cv_openmp.cpp` lê um arquivo texto com a **estatura (cm)** e o **peso (kg)** de 1000 pessoas (uma pessoa por linha, separadas por espaço), agrupa os valores em classes de **8 cm** e de **4 kg** e calcula, para cada variável:

- Média ponderada: $\mu = \Sigma(x_i \cdot f_i) / N$
- Desvio padrão populacional: $\sigma = \sqrt{(\Sigma\, f_i \cdot (x_i - \mu)^2 / N)}$
- Coeficiente de variação: $CV = (\sigma / \mu) \times 100\%$

x_i é o ponto médio da classe i e f_i é a sua frequência. A primeira classe começa no menor valor, arredondado para baixo. Como o arquivo de dados não veio com o enunciado, o programa `gerar_dados.cpp` cria um arquivo com valores realistas (estatura com média 170 cm e desvio de 10 cm; peso obtido de um IMC com média 24). A semente é fixa, então o arquivo é sempre o mesmo.

O programa tem duas versões do mesmo cálculo: `calcular_sequencial()` e `calcular_paralelo()`. Ele confere se as duas dão o mesmo resultado e mede o tempo de cada uma.

2. O que foi paralelizado e por quê

Etapa	Tamanho do laço	Diretiva OpenMP	Por quê
Menor e maior valor	N pessoas	`parallel for reduction(min:menor) reduction(max:maior)`	Todas as threads atualizariam as mesmas duas variáveis. Com a reduction, cada thread usa uma cópia privada e o OpenMP compara as cópias no final.
Frequência de cada classe (soma das pessoas por classe)	N pessoas **(o laço mais pesado)**	`parallel + for schedule(static) nowait + critical`	`freq[classe]++` em paralelo é condição de corrida. Cada thread conta num vetor próprio (`freq_local`) e só no fim soma no vetor geral, dentro de um `critical`. Assim o critical roda poucas vezes por thread, e não uma vez por pessoa. É a mesma ideia da reduction: cada mesário conta os votos da sua urna.
Soma da média $\Sigma\, x_i \cdot f_i$	k classes (~10)	`parallel for reduction(+:soma)`	A soma é a operação típica de reduction. O laço é muito curto (uma iteração por classe), então o ganho aqui é desprezível.
Soma do desvio $\Sigma\, f_i \cdot (x_i - \mu)^2$	k classes (~10)	`parallel for reduction(+:soma_desvios)`	Mesmo motivo. Fica numa região paralela separada porque depende da média, que precisa estar pronta antes.

As outras variáveis do laço não precisam de proteção. Os vetores de entrada são compartilhados, mas só são lidos. O índice `i` e o que é declarado dentro do laço (`classe`, `d`) são privados de cada thread. O `schedule(static)` foi escolhido porque toda pessoa custa o mesmo (uma divisão e um incremento), e por isso não há desbalanceamento a corrigir. A leitura do arquivo ficou sequencial, porque as linhas são lidas uma depois da outra.

3. Resultados obtidos (1000 pessoas)

Estatura (classes de 8 cm)		
classe (cm)	x_i	f_i
[140, 148)	144	17
[148, 156)	152	63
[156, 164)	160	210
[164, 172)	168	293
[172, 180)	176	260
[180, 188)	184	122
[188, 196)	192	32
[196, 204)	200	3
Total N		**1000**

Peso (classes de 4 kg)		
classe (kg)	x_i	f_i
[34, 38)	36	4
[38, 42)	40	3
[42, 46)	44	17
[46, 50)	48	37
[50, 54)	52	59
[54, 58)	56	78
[58, 62)	60	112
[62, 66)	64	103
[66, 70)	68	123
[70, 74)	72	114

Peso (cont.)		
classe (kg)	x_i	f_i
[74, 78)	76	101
[78, 82)	80	96
[82, 86)	84	54
[86, 90)	88	47
[90, 94)	92	21
[94, 98)	96	15
[98, 102)	100	5
[102, 106)	104	5
[106, 110)	108	5
[110, 114)	112	1

Variável	Média μ	Desvio padrão σ	Coeficiente de variação
Estatura	169,80 cm	10,2711 cm	**6,05 %**
Peso	69,18 kg	12,8424 kg	**18,56 %**

O peso varia proporcionalmente **três vezes mais** que a estatura. O CV permite essa comparação porque não tem unidade: cm e kg não podem ser comparados diretamente, mas porcentagens podem. A versão paralela deu exatamente as mesmas frequências e o mesmo CV com 1, 2, 4, 8 e 16 threads.

4. Análise de desempenho

Ambiente: AMD Ryzen 7 5700X3D (8 núcleos físicos, 16 lógicos), Ubuntu 24.04 em contêiner Docker, g++ 13.3 com `-O2 -fopenmp`. Cada tempo é a mediana de 11 execuções, depois de 1 execução de aquecimento descartada. Speedup = tempo sequencial ÷ tempo paralelo. Eficiência = speedup ÷ número de threads.

a) Os 1000 dados do exercício

versão	threads	tempo (ms)	speedup
sequencial	1	0,0036	1,00
paralela	1	0,0076	**0,48**
paralela	2	0,0101	**0,36**
paralela	4	0,0088	**0,41**
paralela	8	0,0124	**0,29**
paralela	16	0,0335	**0,11**

b) Dados repetidos 10 000× (10 milhões de pessoas)

versão	threads	tempo (ms)	speedup	eficiência
sequencial	1	43,25	1,00	100%
paralela	1	40,53	1,07	107%
paralela	2	22,43	1,93	96%
paralela	4	13,89	3,11	78%
paralela	8	10,79	**4,01**	50%
paralela	16	15,24	2,84	18%

Com 1000 pessoas, a versão paralela é mais lenta. O cálculo inteiro leva cerca de 4 microssegundos. Abrir as regiões paralelas, acordar as threads e esperar na barreira custa mais do que isso. Com 16 threads o custo é ainda maior, porque há mais threads para sincronizar.

Para medir o ganho real, os mesmos dados foram repetidos 10 000 vezes (`./cv_openmp pessoas.txt 10000`). Repetir multiplica todas as frequências pelo mesmo número, então **média, desvio e CV não mudam** (o programa confere isso). Só o trabalho aumenta. Com 10 milhões de pessoas, o paralelo chega a **4× com 8 threads**. Ele não chega a 8× porque cada pessoa exige pouquíssima conta (uma divisão e um incremento): o gargalo passa a ser trazer os dados da memória RAM, e não o processador, como na aula

005. Com 16 threads o tempo piora: as threads extras são do hyper-threading (dividem o mesmo núcleo físico) e disputam a mesma memória, então só acrescentam custo de sincronização. Os tempos foram medidos dentro do Docker no Windows, que divide a máquina com o sistema. No laboratório (Linux direto) a variação deve ser menor.

c) Teste dos tipos de paralelização (`speedup_tipos.cpp`): a mesma soma de 10 milhões de estaturas feita de 7 jeitos. O sequencial levou 7,87 ms. A tabela mostra o speedup, e * indica resultado errado.

versão \ threads	1	2	4	8	16	resultado
sem proteção	1,02	1,80*	3,41*	6,46*	1,00*	ERRADO
critical	0,15	0,03	0,01	0,01	0,00	certo
atomic	0,18	0,05	0,02	0,01	0,01	certo
reduction static	0,89	1,86	3,34	**7,74**	5,24	certo
reduction dynamic	0,21	0,08	0,06	0,05	0,05	certo
reduction guided	0,95	1,87	3,40	5,78	6,42	certo

- **Sem proteção**: é rápido, mas a soma sai **errada** com 2 ou mais threads. É a condição de corrida: duas threads leem o mesmo valor e uma soma se perde.
- **critical** e **atomic**: dão a soma certa, mas cada uma das 10 milhões de iterações vira uma fila. Ficam até 100× mais lentos que o sequencial, e pioram com mais threads. O atomic é um pouco melhor que o critical porque usa uma instrução do próprio processador.
- **reduction**: é a única opção certa **e** rápida, porque cada thread soma na sua cópia e só há uma junção no final. O `schedule(static)` é o melhor aqui, já que todas as iterações custam o mesmo. O `dynamic` (pedaços de 1 iteração) passa mais tempo pegando o próximo índice da fila do que somando. O `guided` fica perto do static porque começa com pedaços grandes.

5. Conclusão

Os laços que passam por todas as pessoas (menor/maior e frequências) foram paralelizados com reduction e com contagem local mais critical. As somas da média e do desvio usam `reduction(+)`. O resultado (CV da estatura = **6,05%**, CV do peso = **18,56%**) é idêntico ao da versão sequencial com qualquer número de threads. Com os 1000 dados do enunciado, o paralelismo não compensa, porque o custo de criar e sincronizar threads é maior que o cálculo. Com um volume grande de dados, o speedup chegou a 4× no cálculo do CV e a 7,7× na soma com reduction. Entre os mecanismos de sincronização, a reduction é a escolha certa para somas: o critical e o atomic garantem a corretude, mas transformam o laço numa fila.

Como rodar: `make` (compila e cria `pessoas.txt`) · `make rodar` (1000 pessoas) · `make grande` (10 milhões) · `make tipos` (teste dos tipos).